# VocalIQ — Presentation Script

CS 651: Web Systems | Professor Lynne Grewe | Cal State East Bay

Presented by: **Angaddeep Singh Gupta**

May 5, 2026 · 13 Slides

---

SLIDE 1

## Title Slide

> *"Good morning / afternoon Professor Grewe. My name is Angaddeep Singh Gupta and this is VocalIQ — an AI-powered speaking coach I built for CS651 Project 2. The live application is deployed on Google Cloud Run, and the full source code is on GitHub. Let me walk you through what I built and how it works."*

SLIDE 2

## The Problem

> *"The motivation behind VocalIQ is simple. Most people never get real, actionable feedback on how they speak. Studies show that 75% of the population experiences public speaking anxiety, yet professional speaking coaches charge between $100 and $300 per hour — making personalized coaching completely inaccessible for most people.*
>
> *The core insight is:* **you can't improve what you can't measure.** *VocalIQ fixes this by giving anyone instant, AI-generated coaching after recording a speech — completely free."*

SLIDE 3

## The Solution

> *"VocalIQ works in three steps: you record or upload a speech, our backend processes it through Google Gemini 2.5 Flash, and within 90 seconds you get a full coaching report.*
>
> *The report gives you five sub-scores — Content, Delivery, Confidence, Pacing, and Engagement — plus a timestamped transcript, specific improvement exercises, and a 7-day practice plan. It works for any speaking scenario: job interviews, class presentations, sales pitches, or lectures."*

SLIDE 4

## Technology Stack

> *"On the frontend, I used* **React 19 with Vite** *for fast builds and React Router for client-side navigation.*
>
> *The backend is* **Node.js 20 with Express 5***, using JWT for stateless authentication.*
>
> *For Google services — and this is where the project really comes together — I'm using six Google technologies:*
>
> - **Google OAuth 2.0** *— Sign In with Google*
> - **Gemini 2.5 Flash** *— AI speech analysis*
> - **Cloud Firestore** *— User profiles and session data*
> - **Google Cloud Storage** *— Video and audio files*
> - **Google Cloud Run** *— Containerized deployment*
> - **Google Analytics GA4** *— Frontend and backend event tracking*
>
> *"*

SLIDE 5

## System Architecture

> *"Here's how data flows through the system. The browser runs the React frontend. When a user records a video, the MediaRecorder API captures it. The frontend sends it to the Express API running on Cloud Run, with a JWT token in the Authorization header.*
>
> *The server then kicks off the pipeline: it authenticates the user via the JWT middleware, uploads the video to Gemini's File API, runs the three-pass AI analysis, sends frames to Cloud Vision for emotion detection, saves the video to Cloud Storage, and saves the full session to Firestore.*
>
> *Crucially,* **the frontend never touches Firestore or Cloud Storage directly** *— everything goes through the protected API. This keeps our data secure."*

## SLIDE 6 Three-Pass Gemini Coaching Pipeline ★ KEY SLIDE

*"This is the most technically interesting part of the application — the three-pass Gemini coaching chain.*

**Pass 1 — Assessment:** *The raw video file is uploaded to the Gemini File API. Pass 1 asks Gemini to objectively assess the speech — giving an overall score from 1 to 10, five sub-scores, detected emotion, words per minute, a full timestamped transcript, and the top 3 improvement areas. This pass is purely analytical — no coaching yet.*

**Pass 2 — Deep Coaching:** *Pass 2 takes the JSON output from Pass 1 and feeds it back to Gemini as context. Now Gemini knows exactly what score was given and why. It then generates detailed tip cards with exercises and rephrasing examples, a full vocal control breakdown covering pacing, tone, breathing, and pauses, and a 7-day practice plan.*

**Pass 3 — Gesture Analysis (Drive videos only):** *Pass 3 runs only when the user submits a video from Google Drive. Gemini analyses the video every 3 seconds and returns a gesture label and hand count for each timestamp — open palm, pointing, thumbs up, closed fist, etc. This builds the gesture timeline shown in the Gestures tab of the coaching report.*

**Why three passes instead of one?** *Because if you ask Gemini to do everything in one prompt, it gives generic advice. By grounding Pass 2 in the actual Pass 1 data, the coaching becomes specific to that exact speech. Pass 3 is kept separate because gesture analysis is computationally independent and only relevant for video sessions. The quality improvement is dramatic — this was one of my biggest lessons from this project."*

## SLIDE 7 Authentication — Google OAuth 2.0 + JWT

*"Authentication uses the standard Google OAuth 2.0 flow. The user clicks Sign In with Google, gets redirected to Google's consent screen, Google sends back an authorization code, and our server exchanges that code for the user's profile info.*

*We then upsert the user document in Firestore — so returning users get updated, new users get created — and issue a signed JWT with a 7-day expiry. The JWT is stored in localStorage on the client and sent as a Bearer token on every API request. The authMiddleware verifies it on every protected route.*

*One important security decision:* **we do NOT store Google's access token on the server side.** *We only needed it once to get the user's profile. This minimizes our attack surface."*

SLIDE 8

## Database — Cloud Firestore

*"The database is Cloud Firestore with two collections.*

*The* **users** *collection stores one document per user, keyed by their Google UID — display name, email, photo URL, and last login time.*

*The* **coaching_sessions** *collection stores one document per analysis, keyed by userId_timestamp. Each document has everything: media URL, mode, overall score, all five sub-scores, full transcript, improvement tips, detailed coaching cards, vocal control data, and the practice plan.*

*There's a composite Firestore index on userId ascending and createdAt descending — this lets us efficiently fetch a user's sessions sorted newest-first for the dashboard. All reads and writes go through the JWT-protected API.* **The client never accesses Firestore directly."**

SLIDE 9

## Coaching Report — What You Get

*"The coaching report has five tabs:*

- **Overview** — *Animated score ring with overall score, five sub-scores, and top improvement tips*
- **Emotion Timeline** — *Cloud Vision face detection results: color-coded emotion timeline frame by frame, with an eye contact percentage*
- **Gestures** — *MediaPipe HandLandmarker results: 21 hand landmarks tracked client-side, showing gesture types with an expressiveness score*
- **Transcript** — *Full timestamped transcript with issue tags and suggested fixes for each flagged moment*
- **Tips & Coaching** — *Full Pass 2 Gemini output: detailed tip cards with exercises, rephrasing examples, and the 7-day practice plan*

*This tab structure lets users focus on whichever aspect of their speaking they want to improve."*

SLIDE 10

## Analytics & Logging

*"I implemented analytics at two levels.*

*On the* **frontend**, *gtag.js sends events to GA4 for every meaningful user action: page views on route changes, login success, session submitted, session complete, and which report tab the user clicked.*

*On the* **backend**, *I use the GA4 Measurement Protocol — this sends events directly from the server to GA4 via HTTP POST. This captures events even if the browser closes before analysis finishes. The key backend events are login after the JWT is issued, and session_analyzed after Firestore save — which includes the score, emotion, and source.*

*For logging, every server operation logs to Google Cloud Logging via stdout with a bracketed prefix — [gemini], [coaching], [storage], [analytics] — making it easy to filter logs in the Cloud Run console."*

SLIDE 11

## Deployment — Google Cloud Run

*"The app is deployed on Google Cloud Run using a* **multi-stage Docker build**.

*Stage 1 uses a node:20-alpine image to build the React frontend with npm run build. Stage 2 copies the built React files and the Express server into a clean image. Express serves the React static files AND handles all API routes on the same origin — so there's no CORS issue between frontend and backend.*

*For credentials, I use* **Application Default Credentials (ADC)**. *There's no service account key file in the Docker image. Instead, the Cloud Run service account is granted the necessary IAM roles for Firestore, Cloud Storage, and Gemini. This is the production-safe way to handle credentials on Google Cloud.*

*One lesson learned: Cloud Run assigns the PORT environment variable automatically. You must read process.env.PORT — you cannot hardcode it or the container will never start."*

SLIDE 12

## Lessons Learned

*"Six key lessons from this project:*

- **Express 5 breaking change** — *app.get("*") no longer works for the SPA catch-all. You must use app.use() instead.*
- **Cloud Run PORT** — *Cloud Run injects the PORT variable automatically. You cannot hardcode it or set it yourself. The container simply won't start.*
- **ADC vs key files** — *Never commit a service account JSON file to Git. Use Application Default Credentials on Cloud Run — grant IAM roles to the service account instead.*
- **Vite bakes API URLs at build time** — *Environment variables in Vite are embedded at build, not at runtime. I needed a .env.production file with the correct API base URL before building the Docker image.*
- **GA4 backend events** — *gtag.js only works in the browser. For server-side tracking you need the GA4 Measurement Protocol, which is a separate HTTP API with its own API secret.*
- **Two-pass Gemini prompting** — *One large prompt gives generic coaching. Breaking into Pass 1 (assess) and Pass 2 (coach using Pass 1 data) dramatically improves quality and specificity."*

SLIDE 13

## Thank You / Q&A

*"That wraps up VocalIQ. The app is live right now — feel free to try it. The full source code, wiki documentation, analytics report, and presentation PDF are all on GitHub.*

*I'm happy to do a live demo or answer any questions."*

## ⚡ Likely Professor Questions & Answers

**Q: Why Gemini instead of OpenAI?**

> **A:** *This is a Google Cloud course and Gemini 2.5 Flash has native video understanding via the File API — no need to extract frames manually for the main analysis. It was the right tool for this use case.*

**Q: How does the Gemini File API work?**

> **A:** *You upload the video file to Gemini's File API, which gives back a file URI. You then pass that URI in the prompt content instead of sending base64 data. Gemini processes the video natively — it understands audio, video frames, and speech simultaneously.*

**Q: Why not use Firestore security rules?**

> **A:** *All data access goes through the Express API, which verifies the JWT before any Firestore operation. The API is the security layer — adding Firestore rules on top would be redundant since the client never calls Firestore directly.*

**Q: How does the emotion detection work?**

> **A:** *Every 3 seconds I extract a frame from the video using the HTML5 Canvas API. Those base64 JPEG frames are sent to the Google Cloud Vision Face Detection API, which returns emotion likelihoods — joy, sorrow, anger, surprise — plus head pose angles for eye contact estimation.*

**Q: What happens if Gemini fails mid-pipeline?**

> **A:** *The pipeline is sequential — if Pass 1 fails, we return an error immediately. If Pass 2 fails, Pass 1 data is already returned so the user still gets a partial report. Error handling middleware catches all exceptions and returns consistent JSON error responses.*

**Q: How does Google Drive integration work?**

> **A:** *The user connects their Drive via OAuth with drive.readonly scope. The server downloads the selected video file directly from Drive to a temp file, runs the full 3-pass Gemini pipeline on it including gesture analysis, then cleans up the temp file. The browser never downloads the video — only the session ID is returned to the client.*

**Q: What is MediaPipe and how is it used?**

> **A:** *MediaPipe is Google's on-device ML library. It runs entirely in the browser as a WebAssembly module — no server call needed. It tracks 21 hand landmarks per hand and classifies gestures like open palm, pointing, thumbs up, and closed fist. The results build a gesture timeline shown in the Gestures tab of the coaching report.*

💡 **Presentation Tip:** Keep each slide to about 60–90 seconds. Slide 6 (Three-Pass Pipeline) is your most impressive technical slide — spend the most time here. Always offer a live demo at the end if the professor asks.